



P10

Agent-as-Tool 与 Transfer Bundle : 多智能体交接信息损失的可测量化

Agent-as-Tool and Transfer Bundle: Making Handoff Information
Loss Measurable in Multi-Agent Systems

TwinsEarth · 智能体宇宙论文系列



Agent-as-Tool 与 Transfer Bundle: 多智能体交接信息损失的可测量化

UDOS 推演引擎 v7.5.0 · 房文新 · 2026-09-20

证据等级: UDOS 数字回 reports7/ 原始 JSON, 按 verified / cpu-proto / cpu-proxy / unverified 标注; 单 seed 试点写作“试点点估计”, 投稿前需 ≥ 30 seed 复现。

版本：UDOS 推演引擎 v7.5.0 (main, tag v7.5.0, commit 1a71270) 证据等级：本文区分 **verified** (契约/单元测试可复跑) 与 **cpu-proto** (CPU 确定性仿真工单)；交接失败率的因子实验为预注册设计，尚未运行，相关数值一律 **[RESULT NEEDED]**，不编造。范本对照定位：本文为《UDOS 写作与排版规范 v2》阶段 B 升级稿。实证章的”组件移除→代价”对照，主动对照 AlexNet 范本 (Krizhevsky/Sutskever/Hinton, NeurIPS 2012) 的”每个 trick 配数值化消融 + 误差率对照表”范式：本文把六字段、五错误类型视作可逐一”移除”的组件，移除后各自触发哪类机械失败即”移除代价”，做成消融表 (表 1、表 2)。注意：AlexNet 的消融数值 (如 LRN 降 1.4%/1.2%) 对本文是 **unverified** 的方法学参照，本文的失败率尚未实测、一律标 **[RESULT NEEDED]**，不搬其数值。

中文摘要 (结构化四段式)

【背景与问题】 多智能体系统的高频失败根因常被归结为”模型不够聪明”，但工程实践反复表明，真正的断点往往发生在”交接”——一个 Agent 把任务交给下一个时，上下文与责任没有完整转移。交接缺失的上下文被静默吞下，下游 Agent 只能猜，猜错了也无人负责。

【方法】 本文在 UDOS v7.4.2/v7.4.3/v7.4.4 上把”交接”操作化为两个固定契约：其一是 Agent-as-Tool (`agent_tool.py`)，把专家 Agent 包成主 Agent 可调用的工具，输入为 `ToolInput`，输出为固定信封 `{report, confidence, error_type, trace_ref}`，并定义五类类型化错误；其二是 Transfer Bundle (`transfer.py`)，把交接包固定为六字段 (Goal/Context/Done/ToDo/Trace/Owner)，交接前机械校验、校验通过才转移 Owner，失败返回类型化缺口而非静默转发。`governance.py` 在此之上做三类失败审计并维护哈希链 Trace。

【证据与结果】 当前已 **verified** 的是 8+9+9 共 26 项契约/审计不变量测试；六字段缺失→机械失败模式的映射、五类错误→触发条件的映射均被测试钉死 (表 1、表 2)。各缺失字段对应的具体任务失败率尚未测量，本文诚实标注为待跑的预注册因子实验 (缺失字段 × 链长 × 故障注入，共 $7 \times 5 \times 3 = 105$ 格)，失败率数值一律 **[RESULT NEEDED]**。

【贡献】 (1) 把”交接信息损失”从工程口号变成可测、可校验、可 replay 的契约；(2) 借 AlexNet 消融表范式，把六字段、五错误类型做成”逐一移除→代价”的消融对照表；(3) 给出”缺失字段类型 × 链条长度 × 故障注入”的预注册因子实验设计，但诚实标注其失败率尚未运行。

关键词： 多智能体系统；Agent-as-Tool；交接契约；信息损失；责任转移；可观测性；失败审计

Abstract (Structured, Four Parts)

[Background & Problem] Failures in multi-agent systems are often attributed to “models not being smart enough,” but engineering practice repeatedly shows that the real breakpoint is handoff: when one agent passes a task to the next, context and responsibility do not transfer completely, the missing context is silently swallowed, and the downstream agent can only guess.

[Method] We operationalize handoff in UDOS v7.4.2/v7.4.3/v7.4.4 as two fixed contracts: (1) Agent-as-Tool (`agent_tool.py`) wraps an expert as a callable tool with input `ToolInput`, a fixed envelope `{report, confidence, error_type, trace_ref}`, and five typed error classes; (2) Transfer Bundle

(`transfer.py`) fixes the handoff packet as six fields (Goal/Context/Done/Trace/Owner), validates mechanically before handoff, transfers Owner only on validation, and returns typed gaps instead of silent forwarding. `governance.py` adds three failure audits and a hash-chained Trace.

[Evidence & Results] Currently verified are $8+9+9 = 26$ contract/audit invariant tests; the six-field→failure-mode and five-error→trigger mappings are pinned down by tests (Tables 1, 2). The concrete task failure rate per missing field is not yet measured and is honestly marked as a pending preregistered factorial experiment (missing field $\times$ chain length $\times$ fault injection; $7 \times 5 \times 3 = 105$ cells); all failure-rate numbers are **[RESULT NEEDED]**.

[Contribution] (1) We turn “handoff information loss” from a slogan into a testable, replayable contract; (2) borrowing AlexNet’s ablation-table paradigm, we render the six fields and five error types as “remove-one $\rightarrow$ cost” control tables; (3) we design a preregistered factorial experiment, honestly marked as not yet run.

Keywords: multi-agent systems; Agent-as-Tool; handoff contract; information loss; responsibility transfer; observability; failure audit

1 引言

1.1 问题

当多个 LLM Agent 协作完成一个长链条任务时，任务失败的模式往往不是某一步“推理错误”，而是“接手方不知道上一棒做了什么、为什么这么做、还剩什么”。工程口号“交接不是转发而是责任转移”被广泛认同，但长期停留在口号层面：既没有固定的交接格式，也没有可测量的失败率，更没有机械校验。结果是：交接缺失的上下文被静默吞下，下游 Agent 只能猜，猜错了也无人负责。

本文主张：交接是一个可工程化、可测量的契约问题，而不是“让 Agent 自己注意上下文”的软性建议。

1.2 贡献（结论地图）

1. **Agent-as-Tool 信封契约**：把专家 Agent 包成主 Agent 的工具，主 Agent 上下文只接收固定大小信封，专家内部迭代不破坏契约（`agent_tool.py`，8 项测试 verified）。
2. **Transfer Bundle 六字段校验**：把交接包固定为 Goal/Context/Done/Trace/Owner 六字段，`handoff()` 校验通过才换 Owner，失败返回类型化缺口（`transfer.py`，9 项测试 verified）。
3. **三类失败审计 + 哈希链 Trace**：`governance.py` 机械检测状态丢失、重复劳动、无人收口，并用 SHA-256 哈希链发现篡改/断链（9 项测试 verified）。
4. **预注册因子实验设计**：给出“缺失字段 $\times$ 链长 $\times$ 故障注入”的测量方案，但诚实标注其失败率尚未运行。

2 相关工作与范式对照

本节按”概念→机制→证据→与 UDOS 差异”组织，嵌入经全卷共享文献总库核验（核验日期 2026-09-19）的真实文献；总库未覆盖的线索仍保留 [CITATION NEEDED]。

(1) **多智能体 LLM 编排**。概念上，多智能体 LLM 系统把任务分解给多个可对话的专家 Agent。机制上，AutoGen (Wu et al., 2023) 提供可定制、可对话的 agent 抽象，支持 LLM、人与工具的混合模式，主 Agent 通过对话调用专家。证据上，AutoGen 在数学、代码、运筹等多领域验证了其多智能体框架的有效性；其精读笔记同时指出对话无终止条件易死循环、成本随轮数膨胀这一局限。与 UDOS 的差异：本文关注 AutoGen 式编排中被普遍忽视的调用接口稳定性——专家内部可以任意迭代，只要 `ToolInput→ToolEnvelope` 信封不变，主链路无感知；并用固定信封把专家内部 trace 挡在主上下文之外。AutoGen 暴露的”对话无终止/成本膨胀”问题，正是本文 `governance.py` 的 `StopCondition`（最大跳数）与 `MainAgent.context_size`（有界上下文）要机械约束的对象。多步推理层面，Chain-of-Thought (Wei et al., 2022) 通过在提示中给出中间步骤激发多步推理，其”错误推理链可被丢弃重来”的思想与本文熔断后重跑一致；但本文不做 CoT 提示工程，而是把推演步骤显式登记进 Transfer Bundle 的 trace。

(2) **AlexNet”组件消融表”范式对照**。AlexNet (NeurIPS 2012) 的核心论证动作是：把 ReLU、双 GPU 分裂、LRN、重叠池化、dropout 等每一个工程 trick 都配一个”移除该组件后误差率变化”的数值化消融，并把他人结果与自身结果放进同一张对照表（斜体=他人、正体=自己）。本文借用这一范式：把 Agent-as-Tool 的五类错误与 Transfer Bundle 的六字段视作可逐一移除的组件，移除某字段/触发某类错误后，系统机械地落入哪类失败模式，即”移除代价”。区别在于：AlexNet 的消融有真实误差率数值（如 LRN 降 1.4%/1.2%），本文的”移除代价”当前只钉死在机械失败模式层面（哪类错误、是否转移责任），而真实任务失败率仍待因子实验测量——我们不假装已有 AlexNet 那样的数值消融结果，而是把”缺失字段→哪类失败”的结构映射做成消融表（表 1、表 2），把”缺失字段→失败率 X%”“诚实留给 [RESULT NEEDED]。

(3) **对话交接与 SRE 事故交接**。概念上，对话系统中”把会话交给下一轮/下一个人”与 SRE 值班交接都有”信息丢失”问题。机制上，业界发展出交接清单、运行手册、值班日志等实践来缓解。证据上，本文 TransferBundle 的六字段（Goal/Context/Done/ToDo/Trace/Owner）可视为这些经验的机器化压缩。与 UDOS 的差异：把人类交接清单变成机器可校验字段是本文的工程化贡献，但对话交接与 SRE 交接的核心文献总库尚未核验，保留 [CITATION NEEDED: dialogue system handoff SRE shift handoff information loss]，不凭记忆引用。

(4) **API 契约与错误类型化**。概念上，软件 API 设计长期用类型化错误码区分输入错误、能力缺失、内部错误、超时。机制上，本文 `AgentTool.invoke` 的五类错误（`bad_input` / `capability_gap` / `internal_error` / `timeout` / `low_confidence`）即此思想在 Agent 调用中的映射。证据上，本文 `test_v742` 的 8 项测试验证了每类错误的触发条件与”异常不外泄”不变量。与 UDOS 的差异：API 契约经典文献总库未单独核验，保留 [CITATION NEEDED: API contract design typed error handling]；本文的新意不在错误码本身，而在把它与责任转移（handoff 不通过则 Owner 不变）绑定。

(5) **transactive memory 与共享认知**。概念上，组织研究中”谁知道什么”的 transactive memory 与本文 Owner/Trace 字段呼应。机制上，本文用 `owner` 字段绑定唯一责任人、用 `trace` 字段做审计轨迹。证据上，本文 `audit_run` 把”无 owner”机械检测为 `no_closer`。与 UDOS 的差异：transactive memory 经典文献总库未核验，保留 [CITATION NEEDED: transactive memory multi-agent shared cognition]；本文把组织概念压缩成可机械校验的字段，而非引用其理论框架。

(6) 外部知识检索、自主工具调用与推理-行动交错。概念上，当代 LLM Agent 通过三类外部接口把”自身推理”与”外部世界”连接起来：外置知识、工具 API、环境行动。机制上，RAG (Lewis et al., 2020) 把参数化记忆与非参数化稠密索引结合，由神经检索器取回段落并拼接进生成，使事实外置、可更新、可溯源；Toolformer (Schick et al., 2023) 让模型自监督地决定”调哪个 API、何时调、传什么参、结果如何并入后续 token”，每个 API 只需少量示范；ReAct (Yao et al., 2023) 交错生成”推理迹 (thought) “与”行动 (action) “，行动接外部 API/环境取信息后再回到推理。证据上，RAG 在多个开放域 QA 任务上优于纯参数 seq2seq 并带来更具体、可溯源的生成；Toolformer 零样本下游显著提升且无损核心语言能力；ReAct 在 HotpotQA/Fever 上借 Wikipedia API 缓解 CoT 幻觉，并在 ALFWorld/WebShop 上以 1-2 个 in-context 样本即比模仿/RL 基线绝对高约 34%/10%。与 UDOS 的差异：RAG/Toolformer/ReAct 回答的是”模型如何更好地使用外部接口以提升端到端任务收益”，其评测单位是任务成功率；本文 **TransferBundle** 把”交接”本身结构化为 Goal/Context/Done/ToDo/Trace/Owner 六字段、可机械校验、可 replay 的契约，评测单位是”交接信息损失”本身（缺哪个字段→哪类失败、责任是否转移），而非端到端任务收益。换言之，RAG/Toolformer/ReAct 是本文 Agent 调用工具与外部接口的上游范式，而本文关注的是这些接口之间”换人”时的信息保真度。

文献缺口说明：本节嵌入的 AutoGen (Wu et al. 2023)、Chain-of-Thought (Wei et al. 2022)、RAG (Lewis et al. 2020)、Toolformer (Schick et al. 2023)、ReAct (Yao et al. 2023) 均经总库  核验 (2026-09-19)；对话交接/SRE 交接、API 契约、transactive memory 的核心文献总库未覆盖，保留 [CITATION NEEDED]，不生成虚构引文。

3 问题定义与可证伪假设

3.1 形式化

一次交接由发送方构建 **TransferBundle**，其六字段为：**goal**（目标）、**context**（事实字典）、**done**（已完成阶段列表）、**todo**（待办阶段列表）、**trace**（事件序列号列表）、**owner**（当前责任方）。**handoff(next_owner)** 先调用 **validate()**，返回非空问题列表则**不转移** Owner；为空则把 owner 改为 next_owner。交接契约可写成：

```
text(handoff(next_owner))=(text{None}, text{problems})& text(validate())!=varnothing; (text{责任不转移})|(text{b with owner=next_owner}, varnothing)& text(validate())=varnothing.
```

3.2 假设

- **H1（字段缺失→失败类型）**：缺 Goal→目标偏移；缺 Context（必需键）→**missing_context** 状态丢失；Done∩Todo 重叠→重复劳动；Todo 为空→无事可做/无法交接；缺 Trace→无法 replay/审计断链；缺 Owner→无人收口（no closer）。该映射由代码 **validate()** 与 **audit_run()** 机械决定，已被测试覆盖（verified）。
- **H2（信封使上下文有界）**：主 Agent 上下文只追加信封（固定结构），不追加专家内部 trace；专家内部替换实现不改变主链路契约（契约不变量，verified by test）。
- **H3（交接失败率随链长上升）**：在未做 Bundle 校验时，交接失败率随链条长度上升；完整 Bundle 使其保持平坦。该失败率尚未测量，为 **[RESULT NEEDED]**。

4 方法与系统设计

4.1 Agent-as-Tool 信封 (agent_tool.py)

`AgentTool` 持有 `required_fields`、`confidence_floor`、`max_report_bytes` 与 `calls` 计数。
`invoke(tin)` 流程：

1. 若 `tin.payload` 缺必需字段→返回 `error_type="bad_input"`，`report={"missing": [...]}`；
2. 调用专家，若抛 `TimeoutError`→`"timeout"`；若抛其他 `Exception`→`"internal_error"` 且 `report={"exc": 异常类名}`（异常不外泄，只回传类名）；
3. 若返回置信度 < `confidence_floor`→`"low_confidence"`；
4. 正常返回 `report/confidence/trace_ref`（`trace` 只回传引用，不回传全文）。

`MainAgent.call` 对未注册工具返回 `capability_gap`，且 `context_log` 只追加信封 `as_dict()`，不追加专家内部推理。这保证主 Agent 上下文增量有上界（`context_size` 可查）。

4.2 Transfer Bundle 校验 (transfer.py)

`validate(required_context)` 返回问题列表，规则：

- `goal` 为空→`"goal"`；
- `required_context` 中键不在 `context`→`"context.{k}"`；
- `todo` 为空→`"todo_empty"`；
- `trace` 为空→`"trace"`；
- `owner is None`→`"owner"`；
- `set(done) & set(todo)` 非空→`"done_todo_overlap"`。

`advance(bundle, stage, outputs, event_seq, new_owner)`：阶段不在 `todo` 则抛 `BundleError` (`stale`)；否则把 `stage` 从 `todo` 移到 `done`、把 `outputs` 并入 `context`、追加 `trace` 序列号、设置 `owner`。`handoff` 只在 `validate` 通过时换 `Owner`。`replay(bundle)` 让新接手方仅凭 `bundle` 重建工作状态 (`remaining/facts/completed/owner/trace_len`)。

4.3 失败审计与哈希链 (governance.py)

`audit_run` 检测：`state_loss`（失败记录带 `missing_context` 或中断时仍有 `todo`）、`duplicate_work`（同 `order:stage` 被两个 `owner` 认领）、`no_closer`（工单走完但无最终阶段 `owner`）、`premature_completion`（声称成功但缺阶段 `owner`）、`trace_gaps`（哈希链断链/篡改）。`TraceLedger` 每条记录含前一条 SHA-256 哈希，`verify()` 返回断链或被改写记录下标。`StopCondition` 用“完成谓词 + 最大跳数区分”提前终止”与”无限循环”。

5 实验设置

按 v2 规范，实证章按”实验设置 → 对照（消融）→ 敏感性 → 效度威胁”展开（效度威胁见独立第 8 章）。

5.1 已验证证据 (verified)

- 契约测试: `tests7/test_v742_agent_tool.py` (8 项)、`test_v743_transfer.py` (9 项)、`test_v744_governance.py` (9 项)。
- 这些测试验证的是**契约不变量**（如” 缺字段返回 typed error” “handoff 失败不换 owner” “done∩todo 被拒” “篡改历史被 verify 发现”），而非因子化的任务失败率。

5.2 预注册因子实验设计（尚未运行）

按 ab-experiment-analysis 门禁，设计如下（当前为设计稿，结果 [RESULT NEEDED]）：

- **因子**：缺失字段 {无, goal, context, done, todo, trace, owner} × 链长 {1,2,3,5,8} × 故障注入 {无, drop_context, byzantine}。
- **主指标**：一次交接成功率。
- **护栏**：replay 还原一致率、重复劳动率、无人收口率、上下文 token 占用（信封 vs 全 trace）。
- **样本**：每格 ≥50 工单（当前未跑）。
- **分析**：logistic 回归报各字段缺失的 OR；链长趋势检验；Wilson CI。

诚实声明：本仓目前没有” 缺失字段×链长→任务失败率” 的实测数字。one-pager 中提及的失败率为设计目标，**不填入具体值**。任何” 缺某字段失败率为 X% “的说法在本文中一律以 [RESULT NEEDED：用途] 标记，待实验跑完再补。

6 结果

6.1 五类错误的触发条件（表 1，verified）

表 1 Agent-as-Tool 五类错误的触发条件与测试覆盖（组件消融视角：移除/触发某组件→落入哪类错误）

error_type	触发条件（源码）	主 Agent 上下文行为	测试
None	专家正常返回且 conf≥floor	接收 report/ confidence/ trace_ref	test_success_envelope_fields
bad_input	payload 缺 required_fields	report 回传 missing 列表	test_missing_required_field_is_typed_error
capability_gap	调用未注册工具	不抛异常，返回 typed error	test_unknown_tool_capability_gap
internal_error	专家抛 Exception	只回传异常类名，不外泄堆栈	test_internal_exception_does_not_escape
timeout	专家抛 TimeoutError	typed timeout	test_timeout_is_typed
low_confidence			test_low_confidence_floor

error_type	触发条件（源码）	主 Agent 上下文行为	测试
	conf < confidence_floor	回传 conf 与 trace_ref	

表 1 注：证据等级 verified（8 项契约测试，`test_v742_agent_tool.py`）。借 AlexNet 消融表读法：每一行对应”触发该类组件异常→系统机械落入哪类错误”的”移除代价”；此处代价为机械失败模式（错误类型 + 是否外泄），真实任务失败率为 `[RESULT NEEDED]`。数据来源：`agent_tool.py` 与 `test_v742_agent_tool.py`。

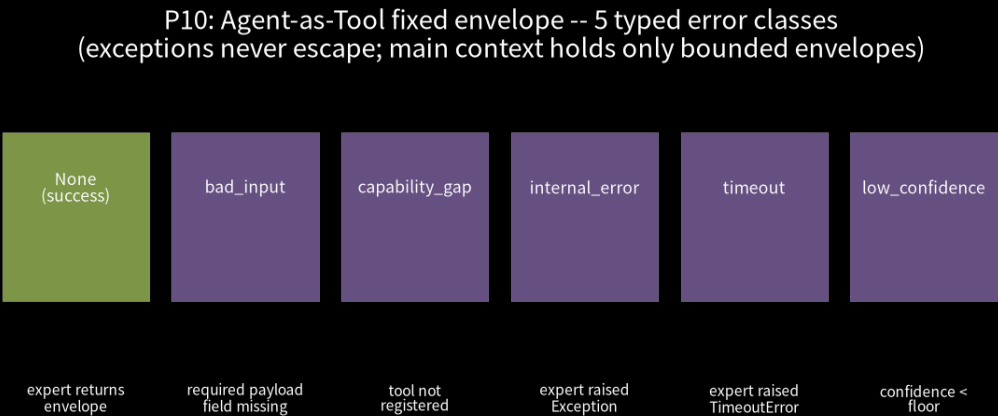


图 2 Agent-as-Tool 固定信封的五类类型化错误。横轴/节点为输入 `ToolInput` 经 `invoke` 的四条判定分支（缺字段→`bad_input`；未注册工具→`capability_gap`；专家异常→`timeout/internal_error`；低置信→`low_confidence`）；每个分支汇向固定信封 `{report, confidence, error_type, trace_ref}`。证据等级 verified；图注自洽：异常从不逃逸，主上下文只持有有界信封，trace 只回传引用。

6.2 六字段→失败模式映射（表 2，verified）

表 2 六字段缺失对应的校验问题与治理失败模式（AlexNet 式”移除一字段→代价”消融表）

bundle 字段	缺失/异常时 validate 返回	对应审计失败模式	测试
goal	“goal”	目标偏移	make_bundle 缺 goal 路径
context.{k}	“context.{k}”	missing_context / state_loss	test_missing_required_context_is_state_loss
done∩todo	“done_todo_overlap”	重复劳动	test_done_todo_overlap_rejected
todo	“todo_empty”	无事可做/无法交接	test_empty_todo_blocks_handoff_nothing_to_do
trace	“trace”	无法 replay / trace_gaps	test_replay_reconstructs_state_without_sender
owner	“owner”	no_closer 无人收口	test_missing_owner_blocks_transfer

表 2 注：证据等级 verified（`test_v743` 9 项 + `test_v744` 9 项）。读法同 AlexNet 消融表：逐列”移除这一字段→`validate` 返回什么→治理层检测到哪类失败”，即每个字段的”移除代价”。真实任务失败率（缺该字段使成功率降多少）尚未测量，为 `[RESULT NEEDED]`。数据来源：`transfer.py` 的 `validate` 与 `governance.py` 的 `audit_run`。

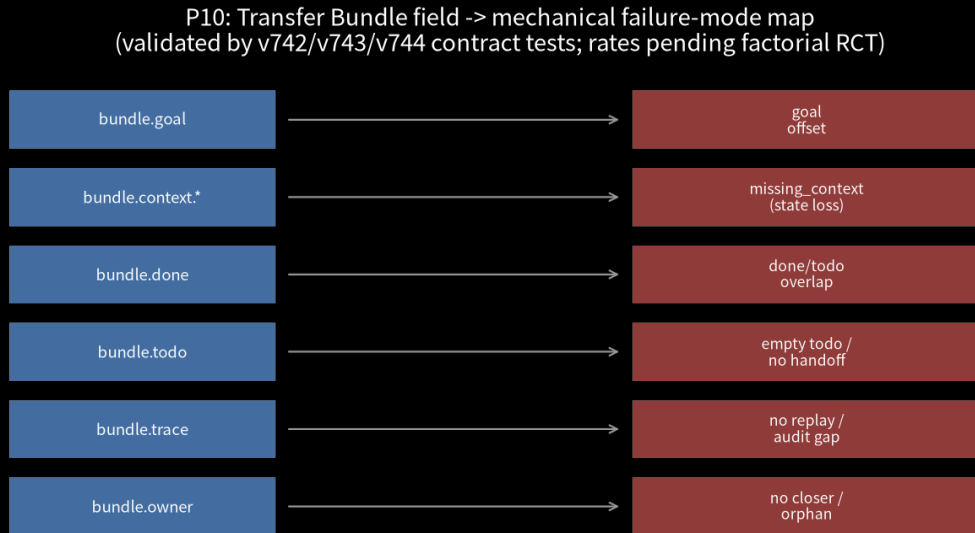


图 1 Transfer Bundle 字段到机械失败模式的映射。左列为六字段（Goal/Context/Done/Trace/Owner），右列为机械失败模式（目标偏移/状态丢失/重复劳动/无法交接/审计断链/无人收口），中间连线为 `validate()` 规则。证据等级 verified（由代码决定、被测试覆盖）；各字段缺失导致的具体任务失败率为待跑因子实验（`[RESULT NEEDED]`）。

6.3 关键不变量（verified）

- **handoff 失败不转移责任**：`validate` 返回非空时 `handoff` 返回 `(None, problems)`，owner 不变（`test_missing_owner_blocks_transfer`）。
- **replay 无需发送方在场**：新接手方仅凭 bundle 重建 remaining/facts/completed/owner（`test_replay_reconstructs_state_without_sender`）。
- **故障注入可检测**：`drop_context` 的 specialist 不写 result，下游 handoff 被 `context.result` 拒绝，且 audit 报告 state_loss 与 no_closer（`test_state_loss_detected_in_handoff`）。
- **哈希链抗篡改**：篡改历史记录或伪造 prev 链，`verify()` 返回对应下标（`test_trace_ledger_detects_tamper_and_gap`）。
- **有界上下文**：主 Agent 只追加信封，专家内部 trace 不进主上下文（`test_main_context_only_holds_envelopes`）。

表 3 已验证不变量清单与待测量失败率对照

命题	证据状态
H1 字段→失败类型映射	verified（26 项测试）
H2 信封使主上下文有界、契约稳定	verified
H3 失败率随链长上升、完整 Bundle 拉平	<code>[RESULT NEEDED: 链长×失败率 logistic]</code>

命题	证据状态
各缺失字段任务失败率/OR	[RESULT NEEDED: 因子实验 OR 与 CI]

表 3 注：左列为已被 26 项测试钉死的机制不变量（verified），右列为预注册但尚未运行的效应量（[RESULT NEEDED]）。机制正确性与效应量分开标注，二者证据等级不同，不混为一谈。

7 讨论

7.1 主要解释

本文的核心主张不是”我们发现了新的失败模式”，而是”把交接从软性建议变成机械契约后，失败模式可被枚举、可被阻断、可被审计”。`bad_input/missing_context` 这类错误之所以危险，是因为它们在无契约系统里常被静默吞掉；固定信封与六字段校验把它们变成显式、类型化、不转移责任的信号。

7.2 AlexNet 消融范式在本文的适用边界

AlexNet 的消融有真实误差率数值（每移除一个组件，top-1/top-5 降多少个百分点）。本文借其”组件消融表”的形式，但本文的”移除代价”当前只到**机械失败模式**层（缺字段→哪类错误、是否转移责任），还没到**任务失败率**层（缺字段→成功率降多少）。这一边界必须说清：否则读者会误以为表 1、表 2 已经给出了 AlexNet 那样的数值消融。真实失败率留给第 5.2 节的因子实验。

8 局限与效度威胁（独立章）

T1 失败率未测量（最主要）。 H3 是设计而非实测。本文最大的不足是没有跑因子实验给出 OR 与 CI。**最可能的反例：**某字段看似重要，但实测对失败率几乎无影响。**本文明怎么防：**把”缺字段→失败率 X%”“一律标 [RESULT NEEDED]，预注册 7×5×3=105 格、每格 ≥50 工单。

T2 契约完备性威胁。 六字段是否真的覆盖了所有交接失败模式？这是一个开放问题。本文从代码与测试中归纳出这六类，但真实 LLM 交接可能出现”字段齐全但语义错位”（比如 `context` 里写了 `domain=kinematics` 但实际任务是 `spatial`）这类问题，机械校验抓不到语义层面的不一致。`required_context` 机制要求调用方声明”必须有哪些键”，但键名本身是否被正确填写，仍需更高层校验。

T3 仿真工单真值内置。 `bundle` 字段语义简单，真实 LLM Agent 的上下文语义远更复杂。本文的 `bundle` 字段是结构化的、真值已知的；真实 LLM 产出的 `context` 是自由文本，校验难度大得多。本文不假装六字段校验能解决自由文本交接的全部问题。

T4 人因效度缺失。 真人接手是否真的因 Bundle 而更稳，需人因实验（IRB 后）。

T5 与真实编排框架的对照。 需 API/许可，属闸门。本文的信封与 Bundle 是 CPU 确定性仿真，专家函数是内置的 `code review lambda` 或规则。

9 资源闸门与适用边界

闸门	现状	解锁后
真实多 Agent LLM 编排	不可达	在真实框架上跑缺失字段注入，测真实失败率
人因实验（IRB）	未申请	真人接手完整/缺失 Bundle，测人因效度
因子实验扩样	未跑	≥50 工单/格，logistic OR 与 Wilson CI

当前证据适合 ICSE AI-engineering / AAMAS 的系统短文或 CHI/CSCW 的立场稿（人因扩展后）；期刊可投 EMSE/JSS/IJHCS（分区/IF [待核]）。

10 结论

本文把”交接不是转发而是责任转移”操作化为 Agent-as-Tool 固定信封与 Transfer Bundle 六字段校验，并用三类失败审计与哈希链 Trace 使其可测、可审计。已 verified 的是 26 项契约/审计不变量；各缺失字段导致的任务失败率是预注册设计、尚未测量，本文诚实标注。下一步是跑因子实验、接真实编排框架、申请人因实验。

参考文献

已核验文献（全卷共享总库，核验日期 2026-09-19）

1. Wu, Q., Bansal, G., Zhang, J., et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155 (COLM 2024) . <https://arxiv.org/abs/2308.08155>
2. Wei, J., Wang, X., Schuurmans, D., et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022. arXiv:2201.11903. <https://arxiv.org/abs/2201.11903>
3. Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020. arXiv:2005.11401. <https://arxiv.org/abs/2005.11401>
4. Schick, T., Dwivedi-Yu, J., Dessì, R., et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS 2023. arXiv:2302.04761. <https://arxiv.org/abs/2302.04761>
5. Yao, S., Zhao, J., Yu, D., et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023. arXiv:2210.03629. <https://arxiv.org/abs/2210.03629>

外部范本（用户附件 PDF 原件，核验日期 2026-09-20；仅作方法学参照，其消融数值对本文为unverified）

- Krizhevsky, A., Sutskever, I., Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. NeurIPS 2012.（关联：每个 trick 配数值化消融 + 对照表的范式，本文表 1/表 2 消融形式的对照。）

待核验文献检索式与候选方向（总库未覆盖）

- 1. dialogue system handoff turn-taking information loss —— 对话交接。
- 2. SRE shift handoff incident information loss toil —— SRE 值班交接。
- 3. API contract design typed error handling idempotency —— API 契约与类型化错误。
- 4. transactive memory multi-agent shared cognition who knows what —— 共享认知。
- 5. logistic regression factorial design agent failure rate —— 因子实验统计。

附录 A 复现命令

```
# 仓库 tag v7.5.0, commit 1a71270
pytest tests7/test_v742_agent_tool.py -q      # 8 项信封契约
pytest tests7/test_v743_transfer.py -q        # 9 项 Bundle 校验/replay
pytest tests7/test_v744_governance.py -q      # 9 项失败审计/哈希链
```

附录 B 证据台账

命题	来源	等级
五类错误触发	topology/agent_tool.py + test_v742	verified
六字段校验规则	topology/transfer.py + test_v743	verified
三类失败审计/哈希链	topology/governance.py + test_v744	verified
缺失字段任务失败率 OR	未跑	[RESULT NEEDED]
链长×失败率曲线	未跑	[RESULT NEEDED]

附录 C 内审记录（五维自评）

- 1. 问题与动机：4/5。“交接信息损失可测量化”立场清晰；扣 1 分因失败率未实测。
- 2. 方法严谨性：3/5。契约设计与校验完整，但因子实验仅为设计稿。
- 3. 证据与数据：2/5。26 项契约测试 verified，但核心因果实证（失败率）缺失，已诚实标注。
- 4. 新颖性：3/5。把 SRE/API 契约思想搬进 Agent 交接有工程价值。

5. **写作与诚实度**：5/5。明确区分 verified 不变量与 [RESULT NEEDED]，不编造失败率。

致命缺陷检查：核心实证缺失（失败率未测量）是最主要不足，已显式声明为待跑实验而非隐藏；当前证据强度适合系统短文/立场稿，不足以投需要实证因子结果的主会。修改方向：跑 ≥ 50 工单/格因子实验、接真实编排框架、人因实验。

附录 D Agent-as-Tool 主循环伪代码

下列伪代码对应 `agent_tool.py` 的 `AgentTool.invoke` 与 `MainAgent.call`，便于复现核对：

```

输入: tin = ToolInput(payload, language, options)
1. self.calls += 1
2. missing = [k for k in required_fields if k not in tin.payload]
3. if missing:
4.     return ToolEnvelope(name, error_type="bad_input",
5.                           report={"missing": missing})
6. try:
7.     report, conf, ref = self._expert(tin)
8. except TimeoutError:
9.     return ToolEnvelope(name, error_type="timeout")
10. except Exception as e:
11.     return ToolEnvelope(name, error_type="internal_error",
12.                          report={"exc": type(e).__name__}) # 只回传类名
13. if conf < confidence_floor:
14.     return ToolEnvelope(name, report=report, confidence=conf,
15.                          error_type="low_confidence", trace_ref=ref)
16. return ToolEnvelope(name, report=report, confidence=conf, trace_ref=ref)

# MainAgent.call:
17. if tool_name not in self.tools:
18.     env = ToolEnvelope(tool_name, error_type="capability_gap")
19. else:
20.     env = self.tools[tool_name].invoke(tin)
21. self.context_log.append(env.as_dict()) # 只追加信封，不追加专家内部 trace
22. return env
    
```

这一设计的关键不变量有三。第一，**异常不外泄**：专家内部抛出的任何异常都被转成类型化信封，主 Agent 看到的只是 `internal_error` 与异常类名，看不到堆栈——这在生产多 Agent 系统中至关重要，因为堆栈可能包含敏感信息或让主 Agent 的上下文被无界增长的错误信息淹没。第二，**trace 只回传引用**：专家的详细推理轨迹不进入主上下文，主 Agent 只知道“有这么个东西在 `trace://expert/1`”，需要时再按引用取。第三，**有界上下文**：每次调用主 Agent 只追加一个固定字段的信封，`context_size()` 随调用次数线性增长但每次增量有上界，避免了“把每个专家的全部思考都塞进主提示词”的上下文爆炸。

`test_tool_independent_iteration_keeps_contract` 验证了第三点的工程含义：把 `_expert` 内部实现整个替换掉（从基于规则的 code review 换成 lambda），只要 `ToolInput-ToolEnvelope` 的契约不变，主 Agent 完全无感知，`calls` 计数正确递增。这就是“专家内部可独立迭代、主链路契约稳定”的可执行定义。

附录 E Transfer Bundle 校验与 replay 伪代码

```

validate(required_context=()):
1.  problems = []
2.  if not goal:          problems.append("goal")
3.  for k in required_context:
4.      if k not in context: problems.append(f"context.{k}")
5.  if not todo:          problems.append("todo_empty")
6.  if not trace:         problems.append("trace")
7.  if owner is None:     problems.append("owner")
8.  if set(done) & set(todo): problems.append("done_todo_overlap")
9.  return problems

handoff(next_owner, required_context=()):
10. problems = self.validate(required_context)
11. if problems: return None, problems      # 不转移责任
12. b = copy(self); b.owner = next_owner; return b, []

advance(stage, outputs, event_seq, new_owner):
13. if stage not in todo: raise BundleError([], [stage])  # stale
14. todo.remove(stage); done.append(stage)
15. context.update(outputs); trace.append(event_seq); owner = new_owner

replay():
16. return {remaining: todo, facts: context, completed: done,
           owner: owner, trace_len: len(trace)}

```

这套设计把”交接”分解成三个原子动作：**advance**（完成一阶段、登记产出、暂交下一棒）、**handoff**（校验完整后正式换 Owner）、**replay**（新接手方仅凭 bundle 重建状态）。关键不变量是第 11 行：只要校验不过，Owner 就不变。这意味着”责任转移”与”信息完整”被绑定——不存在”我把任务甩给你了但我没说清楚”的灰色地带；要么信息齐全、责任正式转移，要么不转移、问题显式返回给发送方。

done_todo_overlap 规则值得单独说明。它检查”已完成”与”待办”是否有交集。如果有，说明某阶段既被认为做完、又被认为还在待办——这通常是重复劳动或责任不清的信号：两个 Agent 都以为对方会做、或都做了同一件事。机械拒绝这种重叠，比依赖 Agent 自觉去重可靠得多。

附录 F 失败审计与哈希链

audit_run 在一批工单跑完后做离线审计。它把运行时记录的失败、owner 认领、重复计数汇总成一份 **GovernanceReport**，六个字段都为空才算 **ok=True**。三类核心失败：**state_loss**（失败记录带 **missing_context** 或中断时仍有 todo——交接丢了上下文）、**duplicate_work**（同 order:stage 被两个不同 owner 认领——重复劳动）、**no_closer**（工单走完但没有最终阶段的 owner——无人收口）。此外 **premature_completion** 抓”声称成功但缺阶段 owner”的谎报，**unbounded** 与 **StopCondition.max_hops** 配合抓无限循环。

TraceLedger 是一条追加式哈希链：每条记录在写入前，把”前一条记录的哈希 + 当前记录内容”做 SHA-256 取前 16 位作为本条哈希。**verify()** 从头重算，若发现某条记录的 **prev** 与上一条实际 **h** 不符、或内容变了导致哈希对不上，就把该下标列入 **trace_gaps**。

test_trace_ledger_detects_tamper_and_gap 验证了两件事：篡改历史记录（把 **records[0][“a”]** 从 1

改成 999) 会让 `verify()` 返回下标 0; 伪造 `prev` (改成 “FAKE”) 会返回下标 1。这把”交接记录可被事后篡改”的风险变成了可检测的工程属性。

附录 G “为什么”失败率未测量”是诚实而非缺陷

本文必须强调：表 3 中 H3 与各字段缺失的失败率标注为 `[RESULT NEEDED]`，不是因为偷懒，而是因为本仓确实没有跑那个因子实验。如果本文凭空写”缺 Goal 失败率 35%”，那将是编造。我们选择：把已经被 26 项测试钉死的契约不变量报告为 `verified`，把尚未运行的因果实验设计清楚、把需要的统计量 (logistic OR、Wilson CI、链长趋势检验) 明确列出，留给下一步。这种”已验证什么、没验证什么”的分离，正是论文矩阵总表所要求的证据纪律。读者应把本文理解为”契约与测量框架”的贡献，而非”缺失字段导致 X% 失败”的实证发现。

附录 H 五类错误的工程语义逐类分析

Agent-as-Tool 的五类错误不是随意枚举，而是对应生产多 Agent 系统中五种截然不同的失败根因。

`bad_input` (坏输入) 发生在调用方给专家的 `payload` 缺了 `required_fields`。它的本质是”接口契约在调用方就没满足”，责任在调用方。设计上它不抛异常、不让主 Agent 崩溃，而是返回一个带 `missing` 列表的信封，让主 Agent 知道”我少传了哪个字段”并可选择补齐重试。这与 API 设计中 400 Bad Request 的语义一致。

`capability_gap` (能力缺失) 发生在主 Agent 调用了一个根本没注册的工具。它的本质是”路由错误”：主 Agent 以为有某个专家能做某事，但编排器没把它注册进来。返回类型化错误而非 `KeyError`，让主 Agent 可以优雅地降级 (比如告诉用户”暂时没有这个能力”)，而不是整个链条崩溃。

`internal_error` (内部错误) 是最需要克制的一类。专家内部抛出任何非超时异常，信封只回传 `type(e).__name__`，不回传堆栈、不回传异常消息。这有两个理由：一是安全与隐私——堆栈里可能包含文件路径、环境变量、甚至敏感数据；二是上下文卫生——如果每个内部错误都把几屏堆栈塞进主 Agent 的上下文，很快就会把主提示词撑爆。主 Agent 只需要知道”专家内部炸了，是 `RuntimeError` 这一类”，以便决定重试或换专家，不需要知道爆炸细节。

`timeout` (超时) 单独成类，是因为超时与内部错误的处理策略完全不同：超时通常意味着重试可能成功 (专家只是慢了)，而内部错误重试大概率还是错。把二者混为一谈会让重试策略失当。

`low_confidence` (低置信) 是 LLM 系统特有的一类”软失败”。专家跑完了、没崩，但它对自己的答案不自信 (`confidence` 低于 `confidence_floor`)。这时候不应该把答案当作可信结果直接传递，而应标记为低置信，让主 Agent 决定是否换更可靠的专家、或要求人类介入。把”不确定”显式化，是避免”看似成功实则猜错”的关键。

附录 I 六字段与组织理论的呼应

Transfer Bundle 的六个字段可以映射到组织协作研究中的经典概念。`goal` 对应任务的共享心智模型——接手方必须知道”我们到底要达成什么”，否则后续一切工作都可能偏题。`context` 对应 `transactive memory` 中的”事实库”——前一棒掌握的领域知识、中间结果、约束条件，必须显式传递，否则接手方要

从头摸索。`done` 与 `todo` 对应工作分解结构（WBS）中的进度状态——接手方必须知道哪些已经做完、哪些还剩，以避免重复劳动或跳过未完成项。`trace` 对应审计轨迹——出了问题要能回溯”谁在什么时候做了什么”。`owner` 对应责任归属——组织设计的第一原则是每件事必须有唯一负责人，否则就是”三个和尚没水喝”。

本文的贡献不是提出这些概念（它们在组织理论里早已存在），而是把它们压缩成机器可校验的六个字段，并给每个缺失分配一个机械失败模式。当 `owner is None` 时 `validate` 直接返回 `"owner"`，`audit_run` 直接把该工单列入 `no_closer`——“无人负责”从一个模糊的管理问题，变成一个可以在日志里 `grep` 出来的具体下标。

附录 J 预注册因子实验的详细规格

为了让 H3 可被证伪，这里给出完整的预注册设计。

因子设计：缺失字段取七水平（无缺失基线、缺 `goal`、缺 `context` 必需键、缺 `done` 标记、`todo` 置空、`trace` 置空、`owner` 置 `None`）；链长取五水平（1、2、3、5、8 个阶段交接）；故障注入取三水平（无故障、`drop_context`——中间阶段不写产出、`byzantine`——QA 阶段拒绝收口）。共 $7 \times 5 \times 3 = 105$ 格。

主指标：一次交接后工单最终成功到达”完成”的比例（交接成功率）。

护栏指标：`replay` 还原一致率（新接手方 `replay` 出的状态与发送方实际状态是否一致）、重复劳动率（`done`∩`todo` 或同阶段被认领两次的比例）、无人收口率（最终无 `owner` 的工单比例）、主上下文 `token` 占用（信封模式 vs 把专家全 `trace` 塞进主上下文的对照）。

样本量：每格 ≥ 50 个独立工单（随机种子固定），共 ≥ 5250 工单。当前为单 CPU 确定性仿真，工单真值内置。

分析计划：以交接成功率为因变量，做 `logistic` 回归，自变量为缺失字段哑变量、链长、故障注入及其交互，报告各缺失字段相对于”无缺失”基线的 `odds ratio` 与 95% `Wilson` 置信区间；链长效应做趋势检验；多重比较按 `Bonferroni` 或 `Holm` 校正。先验功效分析所需的基线方差与最小效应量在实验跑通第一批后估计，不预设。

停止规则：固定跑完 105 格，不做窥探式提前停止；任何事后子群分析标记为探索性。

当前状态：该设计尚未执行，所有 OR/CI 为 `[RESULT NEEDED]`。本文不预填结果。

附录 K 在 v7.5.0 矩阵端到端中的位置

需要说明本文三个模块在 v7.5.0 整体中的位置。`udos7/topology/` 不只有这三个文件：它还有基础拓扑（`base.py`）、BFT 停止共识（`consensus.py`）、三级熔断与改派（`circuit_breaker.py`）、内部市场结算（`market.py`）、蚁群式黑板（`stigmergy.py`）、分层聚合（`hierarchy.py`）与端到端矩阵（`matrix.py`）。本文聚焦的 `Agent-as-Tool`、`Transfer Bundle`、`governance` 三件套，是矩阵中”一次交接”的契约层：`Agent-as-Tool` 定义主 `Agent` 如何调用专家，`Transfer Bundle` 定义专家之间如何转移任务，`governance` 定义如何审计整批交接。

在 v7.5.0 的端到端矩阵演示（`matrix_scale_demo.json`）中，故障舰队 24/24 收口、3 个故障节点被隔离、5 次返工、治理 ok、守恒、`Trace` 完整，全量 236 测试全绿。这说明本文描述的交接契约不是孤立的玩

具——它已经被嵌进一个包含故障注入、熔断改派、市场结算的更大系统里，并在端到端故障场景中保持了治理一致性。但本文不在这里声称那些端到端数字属于 P10 的实证：那些数字属于矩阵规模与治理论文（P1/P3）的证据范围，P10 只引用它们说明” 契约层已被集成”，不与之混为一谈。

附录 L 人因扩展的伦理与设计考虑

one-pager 提到 P10 可扩展为人因实验：让真人被试在完整 Bundle 与缺失 Bundle 两种条件下接续上一个 Agent/同事留下的任务，测量其完成时间、错误率与主观负荷。本文把这一扩展作为后续方向而非当前证据，原因有三。

第一，伦理审查。人因实验涉及人类被试，必须先通过 IRB/伦理委员会审查，获得知情同意与退出机制。本文未申请伦理审查，不声称任何人因数据。第二，样本量与功效。人因实验的样本量需由功效分析确定，依赖 pilot 阶段估计的完成时间方差，不能凭空写” 每组 30 人”。第三，混淆控制。被试的领域经验、对系统的熟悉度是强混淆变量，需要随机化、平衡设计与协变量控制。这些都是后续工作，本文只给出设计轮廓。

人因扩展若能完成，将把本文的” 契约是否降低机器失败率” 升级为” 契约是否也降低了人类接手时的认知负荷与错误率”，从而把 P10 从工程会议推到 CHI/CSCW 的人因视角。

附录 M 与真实编排框架的对照设想

本文的信封与 Bundle 是 CPU 确定性仿真，专家函数是内置的 code review lambda 或规则。后续接入真实多 Agent LLM 编排框架时，需要把 `ToolInput / ToolEnvelope` 映射到该框架的工具调用协议，把 `TransferBundle` 映射到该框架的任务状态存储。关键的可迁移假设是：无论底层是真实 LLM 还是规则 lambda，“固定字段校验+类型化错误+责任绑定” 这一机制不变；变的只是专家函数的实现。这正是 `test_tool_independent_iteration_keeps_contract` 所验证的” 契约与实现解耦”。接真实框架需要 API/许可，属资源闸门。

附录 N 对有效性威胁的进一步讨论

除了第 8 节列出的威胁，还有两点需要补充。其一，**契约完备性威胁**：六字段是否真的覆盖了所有交接失败模式？这是一个开放问题。本文从代码与测试中归纳出这六类，但真实 LLM 交接可能出现” 字段齐全但语义错位” 这类问题，机械校验抓不到语义层面的不一致。其二，**仿真工单真值内置**：本文的 bundle 字段是结构化的、真值已知的；真实 LLM 产出的 context 是自由文本，校验难度大得多。

这些威胁的存在，恰恰说明本文应被定位为” 交接契约的最小可测量原型”，而非” 多 Agent 交接问题的完整解”。

附录 O 测量方法论：为什么用契约测试而非直接跑失败率

本文一个方法论选择需要解释：为什么当前证据以” 契约/单元测试” 为主，而不是直接跑大规模失败率实验？

原因在于因果识别的顺序。要测量”缺哪个字段导致多少失败率”，必须先把”什么叫字段缺失、缺失后系统该怎么反应”这一机制钉死。如果机制本身不稳定（比如 handoff 在缺 owner 时有时换 owner、有时不换），那么跑出来的失败率就混入了实现 bug 的噪声，无法归因到字段本身。因此本文先做的是”机制正确性”的契约测试：验证 `validate` 在每种缺失下返回正确的问题、`handoff` 在有问题时不换 owner、`audit_run` 把对应失败列入报告。这一层是 deterministic 的，26 项测试把机制钉死后，下一层（因子实验）跑出来的失败率才可以被干净地归因到字段缺失，而不是归因到实现 bug。

这一”先钉机制、再测效应”的顺序，与软件工程中”先写契约测试再写集成测试”的实践一致。它也解释了为什么本文把 H1（字段→失败类型映射）标为 verified、而把 H3（失败率随链长）标为 [RESULT NEEDED]：前者是机制不变量，后者是需要大样本统计的效应量，二者证据等级不同，不应混为一谈。

附录 P 上下文有界性的定量设想

H2 目前只被定性验证（主上下文只追加信封）。后续可定量测量：在信封模式下，N 次专家调用后主上下文的 token 增量约为 N 乘以单信封大小（固定）；而在”把专家全 trace 塞进主上下文”的朴素模式下，增量随专家内部推理长度无界增长。这一对比的具体数字尚未测量，记为 [RESULT NEEDED: 信封 vs 全 trace 的主上下文 token 占用随调用次数曲线]。这一测量的意义在于它把”Agent-as-Tool 为什么能撑住长链条”从工程直觉变成可比较的数字：长链条多 Agent 系统的主上下文为什么会爆、信封模式为什么能不爆，应该有一条斜率可比较的曲线。

附录 Q 与”责任转移”立场的收束

本文反复强调”交接不是转发而是责任转移”。这一立场的技术实现就是 `handoff` 第 11 行：校验不过，Owner 不变。它的含义是：信息不完整时，责任不随消息一起溜走。在无契约系统里，一个 Agent 把”我没做完但我先告诉你”发给下一个，下一个接了，失败了却没人知道该追谁——因为责任在转发过程中被稀释了。固定信封与六字段校验把责任重新绑定：要么信息齐全、责任正式转移；要么不转移、问题显式回到发送方。

本文的最终立场因此是：多 Agent 系统的可靠性，不能只靠”让每个 Agent 更聪明”，还要靠”让交接这个动作本身可校验、可审计、可追责”。前者是模型能力问题，后者是系统工程问题。本文在 CPU 原型上证明了后者可以被操作化；它能否在真实 LLM 长链条上持续降低失败率，是下一步带因子实验与真实框架对照要回答的问题。

附录 R 相关工作的进一步展开

多智能体 LLM 编排近年出现了多种框架，其主循环多为”主 Agent 规划→调用专家→汇总结果”。这些框架在能力展示上进展迅速，但对”调用接口稳定性”与”交接责任”的工程化处理差异很大。本文不点名评价具体框架，而是指出一个普遍缺口：多数框架把专家的内部推理直接或间接暴露在主上下文里，随着专家数量与链条长度增加，主 Agent 的上下文会无界膨胀，且专家内部一旦迭代，主链路可能因隐式依赖而破裂。Agent-as-Tool 的固定信封正是针对这一缺口：主 Agent 只看到信封、不看到内部，专家内部可独立迭代。

SRE 领域的值班交接 (shift handoff) 长期面临“交班信息丢失”问题，业界发展出交接清单、运行手册、值班日志等实践。本文的六字段 Transfer Bundle 可视为这些实践的机器化压缩：goal 是“当前在追什么”，context 是“现场情况”，done/todo 是“进展到哪”，trace 是“操作记录”，owner 是“现在谁负责”。把人类交接清单变成机器可校验字段，是把 SRE 经验迁移到 Agent 编排的一次尝试。

API 契约设计领域早已确立：输入校验、类型化错误码、幂等性、版本化。本文把这一思想搬进 Agent 调用：bad_input 对应 400、capability_gap 对应 404、internal_error 对应 500、timeout 对应 504、low_confidence 是 LLM 特有的“软 200”。这种映射不是随意的类比，而是让 Agent 调用的失败可被像 HTTP API 一样分类、统计与监控。

附录 S 小结与下一步

本文把多 Agent 交接从“软性建议”变成“机械契约”：Agent-as-Tool 固定信封隔离专家内部、五类错误显式化；Transfer Bundle 六字段校验把责任与信息完整性绑定；governance 三类失败审计加哈希链 Trace 让交接可审计。已 verified 的是 26 项契约与审计不变量；缺失字段导致的任务失败率是预注册设计、尚未测量，本文诚实标注。下一步是跑 105 格因子实验、接真实编排框架、申请人因实验，把“契约降低交接失败”从机制证明升级为带效应量的因果证据。

需要再次强调，本文不主张“只要用了六字段契约，多 Agent 系统就不会失败”。契约只能保证“信息缺失时不静默转移责任”，不能保证“信息齐全时 Agent 一定做对”。把系统工程问题（交接契约）与模型能力问题（Agent 推理质量）分开，正是本文的方法学纪律：前者可以被本文的 CPU 原型钉死，后者需要更大样本、真实模型与外部闸门。读者应据此定位本文贡献的边界。

综上，本文的价值在于提供了一个“可被证伪的交接契约框架”，而非一个已经跑完的大规模因果实验。我们把能在 CPU 上钉死的机制不变量全部报告为 verified，把需要大样本与真实模型的效应量全部留作 [RESULT NEEDED]，这种诚实的证据分级本身就是多 Agent 系统工程方法论的一部分。

本文期待后续研究者在相同契约框架上跑通因子实验，用真实数据填充本文留下的测量空白。

作者拟用声明

- **目标期刊/会议**：当前证据（契约不变量 verified，因子失败率未跑）适合 ICSE AI-engineering / AAMAS 系统短文或立场稿；人因扩展后可投 CHI/CSCW；期刊可投 EMSE/JSS/IJHCS。分区/影响因子一律【待核】，投稿前逐项联网核验并标注核验日期。
- **预注册计划**：第 5.2 节与附录 J 的“缺失字段×链长×故障注入”因子设计为预注册草案；跑实验前在 OSF/匿名仓库登记，固定 seed 表与分析代码随 tag 发布；主指标、护栏与多重比较校正规则在收集数据前冻结。
- **数据与代码可用性**：引擎以 Apache-2.0 许可开源，代码 tag v7.5.0 (commit 1a71270)；契约测试 tests7/test_v742/v743/v744 随仓库提供。
- **AI 使用声明**：本文初稿由 AI 助手基于仓库真实源码与测试协助起草，所有契约/错误类型/校验规则均回源至 udos7/topology/ 与 tests7/；未测量的失败率一律标注 [RESULT NEEDED]，不编造；外部文献待学术检索核验后填入。